Relatório dos Slides — Aula 07

Carlos Eduardo Carfi Silva | RA: 240006632

Slide 2 — Objetivo e ligação com o mundo real

O que eu entendi: Essa aula é sobre entender as regras por trás de como o código roda, pra ele fazer exatamente o que parece que vai fazer — sem surpresas de precedência, conversão ou desvio. Na prática, isso serve pra ler código dos outros sem ser enganado pela indentação, evitar conversões implícitas perigosas (tipo o == do JS e do PHP) e escolher a estrutura de controle mais fácil de ler.

Como eu explicaria: Saber como a linguagem avalia expressões não é só teoria — é segurança. Quando escrevo código, preciso ter certeza de que o computador vai interpretar do jeito que eu pensei, não do jeito que "parece" que vai funcionar.

Os casos reais do slide:

- **2014, "goto fail" da Apple:** uma linha goto fail; duplicada pulava a verificação da assinatura — iOS e macOS passaram a aceitar certificados falsos.
- **2008, tocadores Zune:** no último dia de um ano bissexto, um while nunca terminava e travou todos os aparelhos.
- **PHP:** "0e4..." == "0e8..." dá verdadeiro, então logins que comparavam hashes com == aceitavam senhas erradas.

Esses casos aparecem de novo nos slides 10, 19 e 20. Como uso PHP no dia a dia, isso ficou bem claro pra mim: devo sempre usar ===.

Slide 3 — Roteiro da aula

A aula tem seis blocos: quiz de abertura (0–10 min), expressões (10–30 min), conversões/booleanas/curto-circuito (30–45 min), atribuição (45–55 min), seleção/iteração/goto e comandos protegidos (55–70 min), e o exercício "encontre o bug" (70–80 min). É uma aula exploratória: a turma prevê a saída do código antes de executar, e os exemplos rodam no onecompiler.com.

Slide 4 — Quiz de abertura: o que cada linha imprime?

Cada trecho usa uma regra que aparece depois na aula. Abaixo, as seis perguntas com resposta e explicação.

Pergunta 1 — Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

Resposta: -4 512

Por quê: O ** tem precedência maior que o menos unário, então -2 ** 2 vira -(2**2) = -4. E o ** associa à direita: 2**3**2 é 2**(3**2) = 2**9 = 512.

Pergunta 2 — Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

Resposta: 3345

Por quê: O + vai da esquerda pra direita. 1+2 = 3 (número). Ao somar com "3" vira texto ("33"), e daí pra frente é só concatenação: "334" e depois "3345".

Pergunta 3 — JavaScript

```
console.log("5" - 2, "5" + 2);
```

Resposta: 3 52

Por quê: O - só existe pra números, então "5" vira 5 e dá 3. O + também serve pra texto, então o JS converte o 2 em string e concatena: "52".

Pergunta 4 — C

```
printf("%d\n", 3 > 2 > 1);
```

Resposta: 0

Por quê: O C avalia da esquerda pra direita. (3 > 2) é verdadeiro e vale 1, depois 1 > 1 é falso (0). Não funciona como na matemática.

Pergunta 5 — Python

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

Resposta: padrão True

Por quê: O 0 é falso, e o or devolve o segundo operando quando o primeiro é falso. Já 3 > 2 > 1 no Python é encadeado como (3>2) and (2>1), que é True.

Pergunta 6 — JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

Resposta: true false true

Por quê: 0 == "" é true e 0 == "0" também. Mas "" == "0" é false, porque aí as duas são texto e string vazia é diferente de "0". O == não é transitivo.

Slide 5 — Expressões aritméticas: questões de projeto

O que eu entendi: O slide lista o que quem projeta a linguagem precisa decidir: regras de precedência, associatividade, ordem de avaliação dos operandos, restrições a efeitos colaterais, sobrecarga de operadores e mistura de tipos. Nem toda linguagem segue a matemática da escola.

Como eu explicaria: Se não conheço a precedência e a associatividade da linguagem, posso criar bugs silenciosos — sem erro de compilação, mas com resultado errado.

Slide 6 — Precedência e associatividade na prática

O mesmo código em Python, JavaScript e Java pode dar resultados bem diferentes. Por exemplo: Python arredonda -7 // 2 para baixo (-4) e o Java trunca em direção ao zero (-3); o JS nem tem divisão inteira. No JS, -2 ** 2 dá erro de sintaxe — ele obriga a usar parênteses.

Exemplo interessante: No Excel, =-2^2 dá 4, porque lá o menos unário vence o ^, ao contrário do Python (-4).

Conclusão: Na dúvida, usar parênteses. Eles deixam a intenção clara pro compilador e pra qualquer pessoa que leia o código depois.

Slide 7 — Ordem dos operandos e efeitos colaterais

O problema aparece quando uma expressão usa uma variável e chama uma função que altera essa mesma variável. O Java garante a avaliação da esquerda pra direita e dá 8, enquanto o C e o Go dão 20 — porque o padrão deles não define essa ordem.

Conclusão: Sem efeitos colaterais, a ordem não importa e f(x) + f(x) pode virar 2 * f(x). Com efeito colateral isso quebra, então é melhor evitá-los.

Slide 8 — Operadores sobrecarregados

Sobrecarga é o mesmo símbolo com significados diferentes: o + soma inteiros, reais e concatena texto no Java. O risco é a confusão. C++, C#, Python, Rust e Kotlin permitem criar sobrecargas próprias; o Java não.

Em Python, os métodos __add__ e __mul__ permitem usar + e * com objetos próprios, o que deixa o código bem legível (v1 + v2 em vez de v1.add(v2)). Entendi por que o Java proíbe isso: dá pra esconder muita complexidade atrás de um símbolo simples.

Slide 9 — Conversões de tipos: alargamento, estreitamento e coerção

O que eu entendi: Alargamento é ir de um tipo menor pra um maior (int pra float), geralmente seguro. Estreitamento é o contrário e pode perder dados, por isso pede cast explícito. Coerção é quando o compilador converte sozinho.

O exemplo mais assustador foi o int 123456789 virando float e, ao voltar, virar 123456792 — o Java mudou o número sem avisar. Prefiro o jeito do Rust e do Go, que exigem conversão explícita e evitam esse tipo de surpresa.

Slide 10 — Coerção implícita: JavaScript e PHP

Esse slide mostra as esquisitices do "type juggling": o interpretador tenta adivinhar o que eu quero. O - força número e o + prefere texto. O true vira 1, o null vira 0 e o undefined vira NaN. O "baNaNa" acontece porque +"a" tenta virar número, dá NaN, e o NaN é concatenado como

texto.

PHP: o == trata "0e123" como notação científica ($0 \times 10^{123} = 0$), então dois hashes MD5 que começam com "0e" são considerados iguais — o que permite logins com senha errada. É o caso real do slide 2.

Conclusão: Em JS e PHP, sempre === e !==. Como trabalho com essas linguagens, isso é bem real pra mim.

Slide 11 — Expressões relacionais e booleanas

Cada linguagem decide o que é "verdadeiro". No C não existe tipo booleano: 0 é falso e qualquer outro valor é verdadeiro. No Python, 3 > 2 > 1 encadeia como (3>2) and (2>1) = True, o que é mais intuitivo. Contam como falso no Python: 0, 0.0, "", [], {} e None. Já Java, Go e Rust só aceitam booleano no if — se passar um inteiro, nem compila.

Slide 12 — Avaliação em curto-circuito

A linguagem para de avaliar assim que o resultado já está garantido. Num && com o primeiro lado falso, o resultado já é falso. Num || com o primeiro lado verdadeiro, já é verdadeiro.

Em Java, o && evita acessar lista[lista.length] — quando i chega ao tamanho da lista, a primeira condição falha e o acesso nem acontece. Com s nulo, o && dá false sem chamar s.length(), mas o & (que avalia os dois lados) joga NullPointerException.

Em JavaScript, o ?. para no primeiro null e devolve undefined sem erro. O qtd || 10 dá 10 quando qtd = 0 (porque 0 é falso), enquanto qtd ?? 10 dá 0, porque ?? só substitui null e undefined.

Slide 13 — Sentenças de atribuição

Em C, a atribuição é uma expressão e devolve o valor atribuído. Isso permite o if (x = y) — eu queria comparar, mas ele atribui y a x e testa o valor, e compila. Já cometi esse erro por acidente.

Em Python, a atribuição múltipla (a, b = b, a) troca dois valores sem variável temporária. O operador morsa (:=, Python 3.8+) calcula, guarda em n e testa tudo na mesma linha.

Slide 14 — Atribuição de modo misto

Quando os tipos dos dois lados do = são diferentes, a linguagem decide o que fazer. A parte perigosa do Java é o i += 3.7, que compila porque o += tem um cast escondido: i = (int)(i + 3.7).

Armadilha clássica: double taxa = acertos / total * 100 — com inteiros, dá 0.0, porque a divisão inteira acontece antes do alargamento pro double. O tipo da variável de destino não

muda a conta. Esse caso volta no exercício 6.

Slide 15 — Estruturas de controle: quantas são necessárias?

As três básicas são sequência, seleção (if) e iteração (while). Böhm e Jacopini provaram em 1966 que elas bastam pra qualquer algoritmo — o goto não é necessário. For, switch, do e foreach existem por facilidade de escrita, não dão poder novo.

Slide 16 — Seleção de dois caminhos: o "else pendente"

É a dúvida de qual if o else pertence quando há ifs aninhados sem chaves. Em Java, C, C++ e C#, a indentação é ignorada e o else pertence ao if mais próximo que ainda não tem else. No exemplo do livro: sum == 0 é verdadeiro, count == 0 é falso, então cai no else do if interno e imprime 1 — não -1, como a indentação sugeria.

Só Python e F# levam a indentação em conta; Perl, Go e Rust exigem chaves sempre. Prefiro esse comportamento.

Slide 17 — Seleção múltipla: do switch de C ao match

No switch do C sem break, com index = 1, entra no case 1, cai no case 3 (odd += 1), continua caindo: even += 1 e ainda imprime "Error". Ou seja: odd, even e o erro ao mesmo tempo.

No Java 14+, o -> dispensa break, não cai nos outros casos e devolve o valor direto. No Rust, o match exige cobrir todas as possibilidades: sem o _ o código não compila.

Slide 18 — Laços controlados por contador

O for do C é um laço lógico disfarçado: dá pra alterar a variável dentro do corpo. No exemplo do slide, a saída é 0 3 6 9 — o i += 2 dentro do laço muda o contador de verdade.

No Python, o for entrega o próximo valor do range a cada volta, então alterar i dentro do laço não muda nada. O range(10, 0, -3) imprime 10 7 4 1.

Slide 19 — Laços lógicos, saídas e iteradores

Break rotulado (Java): um break normal só sai do laço onde está. Com o rótulo "externo", sai dos dois laços de uma vez.

for...in vs for...of (JS): o for...in percorre os índices (que viram texto), então 0 + "0" + "1" + "2" = "0012". O for...of percorre os valores e dá 60.

Bug do Zune: no último dia de um ano bissexto, dias vale 366. Entra no while, mas 366 > 366 é falso e nada muda — laço infinito. Todo caminho do laço precisa fazer progresso.

Slide 20 — Desvio incondicional: o goto

O goto pula pra qualquer lugar do programa. Dijkstra escreveu em 1968 "Go To Statement Considered Harmful". C, C++, C# e Go têm goto; o Java não tem.

"goto fail" da Apple: o if sem chaves só protege o primeiro goto fail. O segundo está solto e desvia sempre — a verificação da assinatura nunca roda. O err continua 0 (certificado válido) e a assinatura falsa passa. A indentação enganou todo mundo.

Break, continue e return são gotos restritos e melhoram a legibilidade. Com chaves obrigatórias (Go, Rust) ou gcc -Wall, o bug teria sido pego.

Slide 21 — Comandos protegidos de Dijkstra

No comando protegido, as guardas (condições) decidem qual bloco executa. Se x == y, as duas guardas são verdadeiras e qualquer ramo serve. Isso é a base do CSP de Hoare (1978) e do select do Go.

No exemplo Go com dois canais prontos, o runtime sorteia qual executar. Repetindo 1000 vezes: map[a:505 b:495] — muda a cada execução. Bom pra sistemas concorrentes onde nenhum canal pode ficar sempre em segundo plano.

Slides 22, 23 e 24 — Exercícios: encontre o bug

Exercício 1 — JavaScript — for...in percorre índices, não valores

Saída: "Total: 0012"

Por quê: O for...in percorre as chaves do array, que são string ("0", "1", "2"). O total começa em 0 e vai concatenando os índices em vez de somar os preços.

Correção: Trocar por for...of: for (const p of precos). Aí a saída é "Total: 60".

Exercício 2 — Python — curto-circuito com or e valor 0

Saída: 180.0

Por quê: O 0 conta como falso. Em d = d or 10, o 0 é descartado e vira 10. Eu queria 0% de desconto e esperava 200.0.

Correção: Testar com None: if d is None: d = 10.

Exercício 3 — Java — switch sem break

Saída: "inválido"

Por quê: Sem break, com dia = 2: entra no case 2 ("segunda"), cai no case 3 ("terça") e no default ("inválido"). Vale a última atribuição.

Correção: Colocar break; no fim de cada case, ou usar o switch com -> do Java moderno.

Exercício 4 — C — ponto e vírgula depois do if

Saída: "saldo insuficiente" e "fim" (mesmo com saldo sobrando)

Por quê: O ponto e vírgula depois do if vira um comando vazio. O printf seguinte não faz parte do if e roda sempre.

Correção: Tirar o ponto e vírgula. Usar chaves também deixaria a intenção clara.

Exercício 5 — Go — shadowing com :=

Saída: "dentro: 20" e "fora: 10"

Por quê: O := dentro do if cria um novo x (20) que esconde o x de fora. Quando o bloco acaba, o x original continua 10.

Correção: Usar = em vez de :=: x = x * 2. Aí o "fora" também imprime 20.

Exercício 6 — Java — divisão inteira antes do alargamento

Saída: 0.0%

Por quê: acertos e total são inteiros, então 7/10 = 0 (divisão inteira). Depois 0 * 100 = 0, e só no fim vira double. O tipo da variável de destino não muda a conta.

Correção: Cast antes da divisão: (double) acertos / total * 100. Aí 7.0/10*100 = 70.0.

Slide 25 — Síntese

A tabela do slide resume bem: cada recurso moderno da linguagem existe pra fechar uma brecha antiga. Parênteses pra precedência, === pra coerção, match exaustivo pro switch sem break, for...of pro for...in, chaves obrigatórias pro else pendente.

Mesmo com toda essa ajuda da linguagem, ainda cabe a mim escrever com clareza — parênteses onde tem dúvida, tipos explícitos, comparação estrita.

Slide 26 — Atividades complementares

Expressões em Ruby e Lisp, expressões condicionais, tabelas de desvio do switch, laços em linguagens funcionais, iteradores e blocos de Ruby, e leitura do artigo do Dijkstra. A entrega sugerida é comparar o switch em C, switch -> em Java, switch em Go e match em Rust. Próxima aula: Subprogramas (caps. 9 e 10).

Conclusão geral

O que ficou claro pra mim nessa aula é que toda linguagem tem suas próprias regras pra avaliar expressões e controlar o fluxo, e elas mudam bastante de uma pra outra. Se não conheço essas regras, escrevo código que parece fazer uma coisa e faz outra — e o compilador não avisa.

Em expressões, vi que precedência e associatividade decidem a ordem das contas, que efeitos colaterais podem mudar o resultado dependendo da linguagem, e que a coerção implícita (o 123456789 virando 123456792, o "0e123" == "0e456" dando true) cria bugs que

são difíceis de perceber.

Em controle, sequência, seleção e iteração já bastam pra qualquer coisa. Os problemas vieram de brechas na sintaxe: else pendente, switch sem break, for...in, laço sem progresso (Zune) e goto duplicado (Apple). Os seis exercícios foram bugs que dão saída errada sem erro de compilação, e cada um se encaixava numa regra da aula.

A principal lição que levo é não confiar na intuição: deixar a intenção explícita com parênteses, chaves, comparação estrita e cast de tipo escrito. Como trabalho com PHP e JavaScript, o cuidado com o == e a coerção é o que mais vou aplicar no dia a dia.